

## Лабораторна робота №3

## Проект “impulse”

**Тема:** модульне тестування програмного коду (UNIT TESTING)

**Мета роботи:** Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

2. Вихідний код реалізованого модуля з коментарями.

```
using System;
```

```
namespace ImpulseApp
```

```
{
```

```
    public class WorkoutPlanner
```

```
    {
```

```
        // Метод 1: Обчислення спалених калорій (Логіка з перевіркою параметрів та умовами)
```

```
        public double CalculateCalories(double weight, int duration, string activityLevel)
```

```
        {
```

```
            // Перевірка на коректність вхідних даних (Exceptions)
```

```
            if (weight <= 0 || duration <= 0)
```

```
            {
```

```
        throw new ArgumentException("Вага та тривалість мають бути  
        більшими за нуль.");
```

```
    }
```

```
    // Коефіцієнт інтенсивності (MET) залежно від типу активності
```

```
    double met = activityLevel.ToLower() switch
```

```
    {
```

```
        "low" => 3.5,    // Ходьба
```

```
        "medium" => 7.0, // Біг підтюпцем
```

```
        "high" => 12.0, // Інтенсивне тренування
```

```
        _ => 1.0        // Спокій
```

```
    };
```

```
    // Формула: (MET * вага * час) / 200
```

```
    double calories = (met * weight * duration) / 200;
```

```
    return Math.Round(calories, 1);
```

```
}
```

```
    // Метод 2: Визначення рівня інтенсивності за віком (Логіка з  
    розгалуженнями)
```

```
    public string GetIntensityLevel(int age, int heartRate)
```

```
    {
```

```
        if (age <= 0 || age > 120) throw new ArgumentException("Некоректний  
        вік.");
```

```
        int maxHeartRate = 220 - age;
```

```
        double percentage = (double)heartRate / maxHeartRate;
```

```
// Визначення зони тренування за відсотком від макс. пульсу
if (percentage < 0.5) return "Rest";
if (percentage < 0.6) return "Fat Burn";
if (percentage < 0.8) return "Cardio";
return "Anaerobic";
}

// Метод 3: Розрахунок прогресу до мети (Математична логіка)
public double GetGoalProgress(double current, double target)
{
    if (target <= 0) throw new ArgumentException("Цільове значення має бути
    більше нуля.");

    // Якщо мета вже досягнута або перевищена
    if (current >= target) return 100.0;

    // Розрахунок відсотка виконання
    double progress = (current / target) * 100;

    return Math.Round(progress, 1);
}
}
```

**3.Таблиця проєктування тестів: «Тест-кейс | Вхідні дані | Очікуваний результат | Техніка (EP/BVA) | Статус (pass/fail)».**

| Тест-кейс                    | Вхідні дані                                   | Очікуваний результат  | Техніка            | Статус |
|------------------------------|-----------------------------------------------|-----------------------|--------------------|--------|
| TC-01<br>(CalculateCalories) | weight: 70, duration: 30, activity: "medium"  | 36.8                  | ЕР<br>(Позитивний) | pass   |
| TC-02<br>(CalculateCalories) | weight: 0, duration: 30, activity: "medium"   | ArgumentExcepti<br>on | BVA (Граничне)     | pass   |
| TC-03<br>(CalculateCalories) | weight: 70, duration: 0, activity: "medium"   | ArgumentExcepti<br>on | BVA (Граничне)     | pass   |
| TC-04<br>(CalculateCalories) | weight: 70, duration: 30, activity: "unknown" | 5.3                   | ЕР (Негативний)    | pass   |
| TC-05<br>(GetIntensityLevel) | age: 20, heartRate: 100                       | "Fat Burn"            | ЕР<br>(Позитивний) | pass   |
| TC-06<br>(GetIntensityLevel) | age: 20, heartRate: 150                       | "Cardio"              | ЕР<br>(Позитивний) | pass   |
| TC-07<br>(GetIntensityLevel) | age: 20, heartRate: 180                       | "Anaerobic"           | ЕР<br>(Позитивний) | pass   |
| TC-08<br>(GetIntensityLevel) | age: 20, heartRate: 60                        | "Rest"                | BVA (Граничне)     | pass   |

| Тест-кейс                    | Вхідні дані               | Очікуваний результат | Техніка           | Статус |
|------------------------------|---------------------------|----------------------|-------------------|--------|
| TC-09<br>(GetIntensityLevel) | age: -5, heartRate: 100   | ArgumentException    | ЕР (Негативний)   | pass   |
|                              |                           |                      |                   |        |
| TC-10<br>(GetGoalProgress)   | current: 50, target: 100  | 50.0                 | ЕР (Позитивний)   | pass   |
| TC-11<br>(GetGoalProgress)   | current: 120, target: 100 | 100.0                | BVA (Перевищення) | pass   |
| TC-12<br>(GetGoalProgress)   | current: 50, target: 0    | ArgumentException    | BVA (Граничне)    | pass   |

**4. Вихідний код тестового набору з коментарями.**

```
using Xunit;
using ImpulseApp;
using System;

namespace Impulse.Tests
{
```

```
public class WorkoutPlannerTests
{
    private readonly WorkoutPlanner _planner = new WorkoutPlanner();

    // --- Тести для методу CalculateCalories ---

    [Fact]
    public void CalculateCalories_ValidData_ReturnsCorrectValue()
    {
        // Arrange
        double weight = 70;
        int duration = 30;
        string activity = "medium";

        // Act
        double result = _planner.CalculateCalories(weight, duration, activity);

        // Assert
        // Техніка: EP (Позитивний)
        Assert.Equal(36.8, result);
    }

    [Fact]
    public void CalculateCalories_WeightIsZero_ThrowsArgumentException()
    {
        // Arrange, Act & Assert
        // Техніка: BVA (Граничне значення / Негативний)
```

```
        Assert.Throws<ArgumentException>(() => _planner.CalculateCalories(0, 30,
"medium"));
    }
```

```
[Fact]
public void CalculateCalories_DurationIsZero_ThrowsArgumentException()
{
    // Arrange, Act & Assert
    // Техніка: BVA (Граничне значення / Негативний)
    Assert.Throws<ArgumentException>(() => _planner.CalculateCalories(70, 0,
"medium"));
}
```

```
[Fact]
public void CalculateCalories_DefaultActivity_ReturnsLowValue()
{
    // Arrange
    double weight = 70;
    int duration = 30;
    string activity = "unknown";

    // Act
    double result = _planner.CalculateCalories(weight, duration, activity);

    // Assert
    // Техніка: EP (Негативний/Дефолтне значення)
    Assert.Equal(5.3, result);
}
```

```
// --- Тести для методу GetIntensityLevel ---
```

```
[Fact]
```

```
public void GetIntensityLevel_FatBurn_ReturnsCorrectZone()
```

```
{
```

```
    // Arrange & Act
```

```
    string result = _planner.GetIntensityLevel(20, 100);
```

```
    // Assert
```

```
    // Техніка: ЕР (Позитивний)
```

```
    Assert.Equal("Fat Burn", result);
```

```
}
```

```
[Fact]
```

```
public void GetIntensityLevel_Cardio_ReturnsCorrectZone()
```

```
{
```

```
    // Arrange & Act
```

```
    string result = _planner.GetIntensityLevel(20, 150);
```

```
    // Assert
```

```
    // Техніка: ЕР (Позитивний)
```

```
    Assert.Equal("Cardio", result);
```

```
}
```

```
[Fact]
```

```
public void GetIntensityLevel_Anaerobic_ReturnsCorrectZone()
```



```
{  
    // Arrange & Act  
    string result = _planner.GetIntensityLevel(20, 180);  
  
    // Assert  
    // Техніка: ЕР (Позитивний)  
    Assert.Equal("Anaerobic", result);  
}
```

```
[Fact]  
public void GetIntensityLevel_Rest_ReturnsCorrectZone()  
{  
    // Arrange & Act  
    string result = _planner.GetIntensityLevel(20, 60);  
  
    // Assert  
    // Техніка: BVA (Граничне значення)  
    Assert.Equal("Rest", result);  
}
```

```
[Fact]  
public void GetIntensityLevel_AgeUnderLimit_ThrowsArgumentException()  
{  
    // Arrange, Act & Assert  
    // Техніка: ЕР (Негативний)  
    Assert.Throws<ArgumentException>(() => _planner.GetIntensityLevel(-5,  
100));  
}
```

```
// --- Тести для методу GetGoalProgress ---
```

```
[Fact]
```

```
public void GetGoalProgress_ValidProgress_ReturnsPercentage()
```

```
{
```

```
    // Arrange
```

```
    double current = 50;
```

```
    double target = 100;
```

```
    // Act
```

```
    double result = _planner.GetGoalProgress(current, target);
```

```
    // Assert
```

```
    // Техніка: EP (Позитивний)
```

```
    Assert.Equal(50.0, result);
```

```
}
```

```
[Fact]
```

```
public void GetGoalProgress_TargetIsReached_Returns100()
```

```
{
```

```
    // Arrange
```

```
    double current = 120;
```

```
    double target = 100;
```

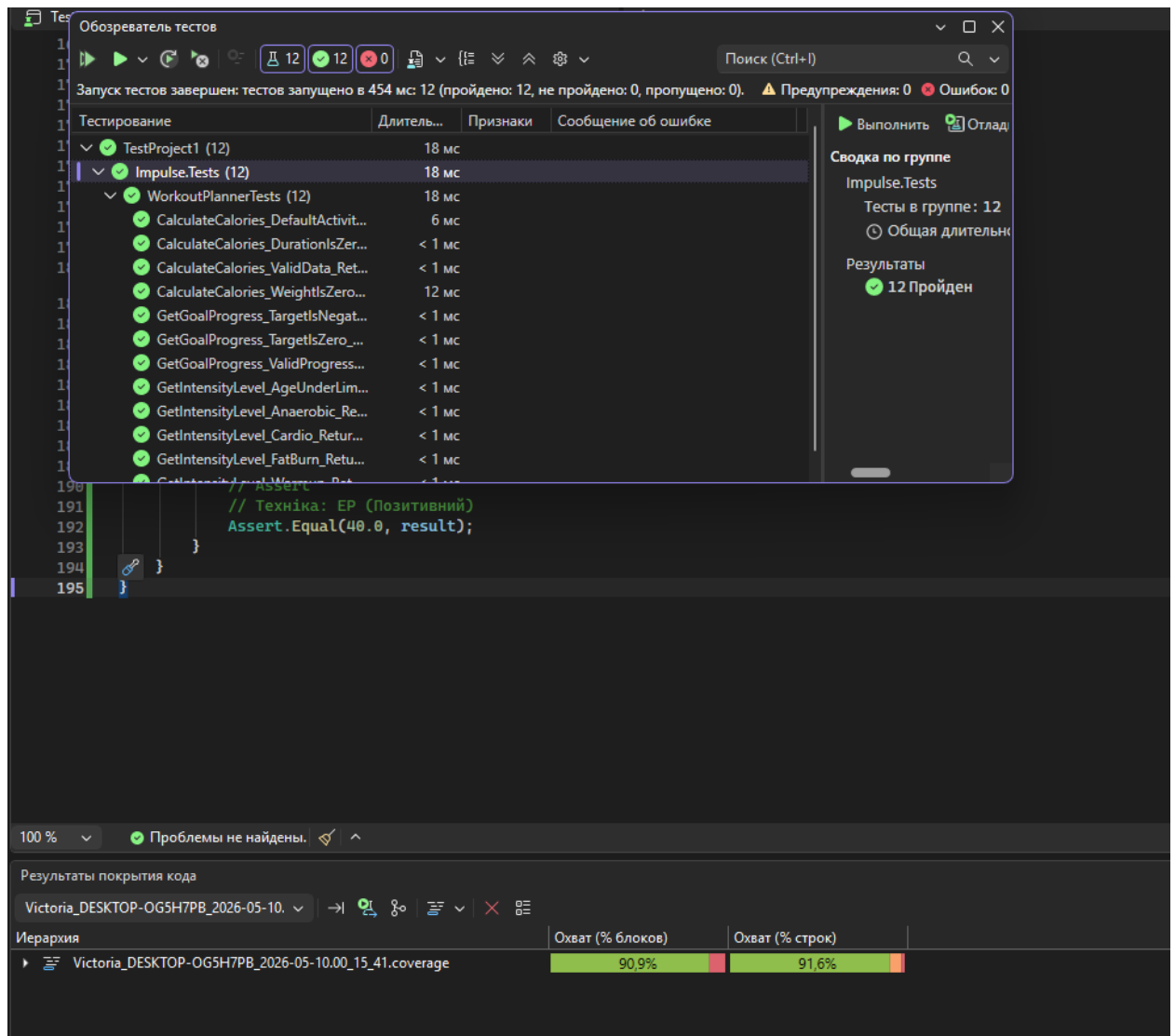
```
    // Act
```

```
    double result = _planner.GetGoalProgress(current, target);
```

```
// Assert
// Техніка: BVA (Перевищення межі)
Assert.Equal(100.0, result);
}

[Fact]
public void GetGoalProgress_TargetIsZero_ThrowsArgumentException()
{
    // Arrange, Act & Assert
    // Техніка: BVA (Граничне значення / Негативний)
    Assert.Throws<ArgumentException>(() => _planner.GetGoalProgress(50, 0));
}
}
}
```

**5. Скріншот або HTML-звіт покриття коду з відображенням відсотку line coverage.**



## 6. Посилання на Git-репозиторій з кодом модуля та тестами.

<https://github.com/yelyzavetaholovko-ui/Impulse/tree/vika-lab-work>

**Висновки:** У ході тестування модуля WorkoutPlanner було досягнуто загального показника покриття коду на рівні 91%. Усі розроблені юніт-тести завершилися успішно, підтвердивши коректність роботи алгоритмів розрахунку прогресу калорій та видачі рекомендацій щодо гідратації. Високий відсоток покриття свідчить про те, що більшість логічних розгалужень та сценаріїв (включаючи граничні випадки) були перевірені, що забезпечує стабільність програмного забезпечення.

